

1 EXERCÍCIO 1: FORK.C

1.1 ANÁLISE DO CÓDIGO

- a) O processo original imprime `Ola, sou o processo 1091` (apenas uma vez, pois o `fork` ainda não ocorreu);
- b) `fork()` cria o filho, que é uma cópia do pai e começa a executar **na instrução seguinte ao fork**. O valor de retorno é o que diferencia os dois: PID do filho no pai, 0 no filho, -1 em caso de erro;
- c) Os dois processos executam o segundo `printf`, em ordem não determinística (depende do escalonador);
- d) O pai chama `wait(0)` e fica bloqueado até o filho terminar; o filho dorme 5 s, imprime `Tchau` e termina com `exit(0)`;
- e) Quando o filho termina, `wait()` retorna no pai, que imprime seu `Tchau` e termina.

Saída real (MSYS2):

Figura 1: Captura de tela da execução de `fork.c` no terminal

```

Asus@DESKTOP-B7L3I96 MSYS /c/Users/Asus/Documents/SO-Laboratório/Lab 2
$ ./fork
Ola, sou o processo 1091
[retval: 1092] sou 1091, filho de 923
[retval: 0] sou 1092, filho de 1091
Tchau de 1092!
Tchau de 1091!
  
```

Fonte: elaboração própria (2026).

(As duas linhas `[retval...]` podem aparecer em qualquer ordem. Os dois `Tchau`, não: o pai só imprime depois do `wait`.)

Observação: se a saída for redirecionada para arquivo (`./fork > saida.txt`), o `Ola` aparece **duas vezes**: o `stdout` passa a ser totalmente bufferizado, e o buffer ainda não esvaziado é copiado para o filho no `fork()`. Execução real com redirecionamento:

Figura 2: Captura de tela da execução de fork.c com a saída redirecionada para arquivo

```

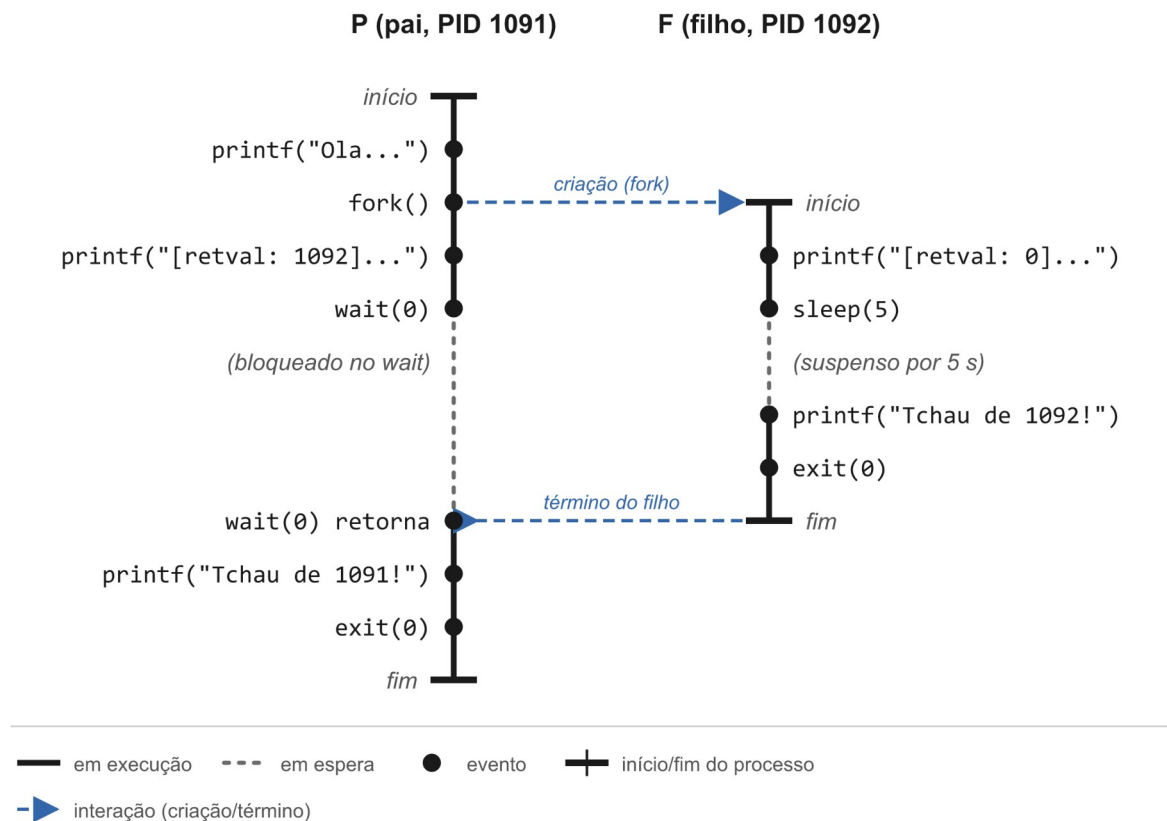
Asus@DESKTOP-B7L3I96 MSYS /c/Users/Asus/Documents/SO-Laboratório/Lab 2
$ ./fork > saída.txt; cat saída.txt
Ola, sou o processo 1179
[retval: 0] sou 1180, filho de 1179
Tchau de 1180!
Ola, sou o processo 1179
[retval: 1180] sou 1179, filho de 923
Tchau de 1179!

```

Fonte: elaboração própria (2026).

1.2 DIAGRAMA DE TEMPO

Figura 3: Diagrama de tempo da execução de fork.c



Fonte: elaboração própria (2026).

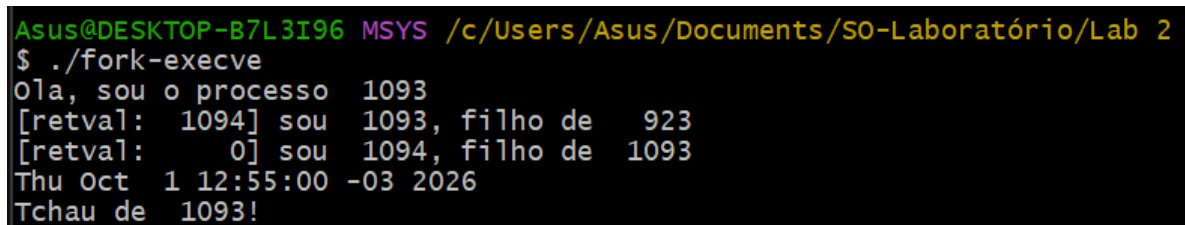
2 EXERCÍCIO 2: FORK-EXECVE.C

2.1 ANÁLISE DO CÓDIGO

- a) Igual ao exercício 1 até o `fork()` e o segundo `printf`;
- b) O filho chama `execve("/bin/date", argv, envp)`, que **substitui a imagem do processo** (código, dados, pilha) pelo programa `/bin/date`. O PID continua 1094, mas o código original deixa de existir nele;
- c) Como `execve` bem-sucedido **não retorna**, o filho nunca executa `perror` nem o `printf("Tchau...")`: quem imprime é o `date`, que depois termina normalmente;
- d) O pai espera com `wait(0)` e, após o término do filho (`date`), imprime `Tchau de 1093!`;
- e) `argv` e `envp` do programa são repassados ao `date` (argumentos extras, como `./fork-execve +%H:%M`, mudariam a saída do `date`).

Saída real (MSYS2):

Figura 4: Captura de tela da execução de `fork-execve.c`



```

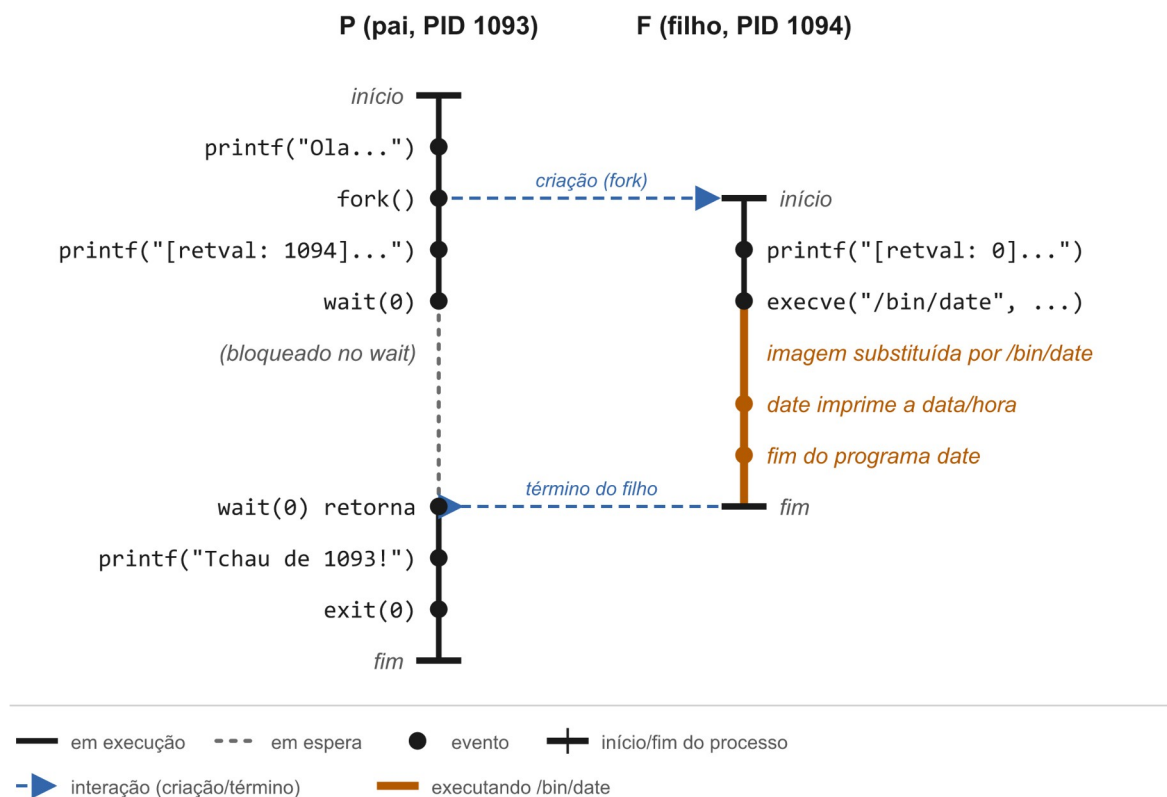
Asus@DESKTOP-B7L3I96 MSYS /c/Users/Asus/Documents/SO-Laboratório/Lab 2
$ ./fork-execve
Ola, sou o processo 1093
[retval: 1094] sou 1093, filho de 923
[retval: 0] sou 1094, filho de 1093
Thu Oct 1 12:55:00 -03 2026
Tchau de 1093!
  
```

Fonte: elaboração própria (2026).

Note que só existe **um** `Tchau`: o do pai.

2.2 DIAGRAMA DE TEMPO

Figura 5: Diagrama de tempo da execução de fork-execve.c



Fonte: elaboração própria (2026).

2.3 E SE O PROGRAMA A SER EXECUTADO NÃO EXISTIR?

`execve()` **falha e retorna -1**, com `errno = ENOENT`. A imagem do filho não é substituída, então ele continua executando o código original: `perror("Erro")` imprime `Erro: No such file or directory`, depois o filho executa o `printf("Tchau...")` e o `exit(0)`, ou seja, o filho passa a fazer o mesmo que o pai. O `wait()` do pai retorna e ele também imprime seu `Tchau`.

Saída real (programa `fork-execve-falha.c`, que é o `fork-execve.c` com `/bin/date` trocado por `/bin/naoexiste`):

Figura 6: Captura de tela da execução de fork-execve-falha.c

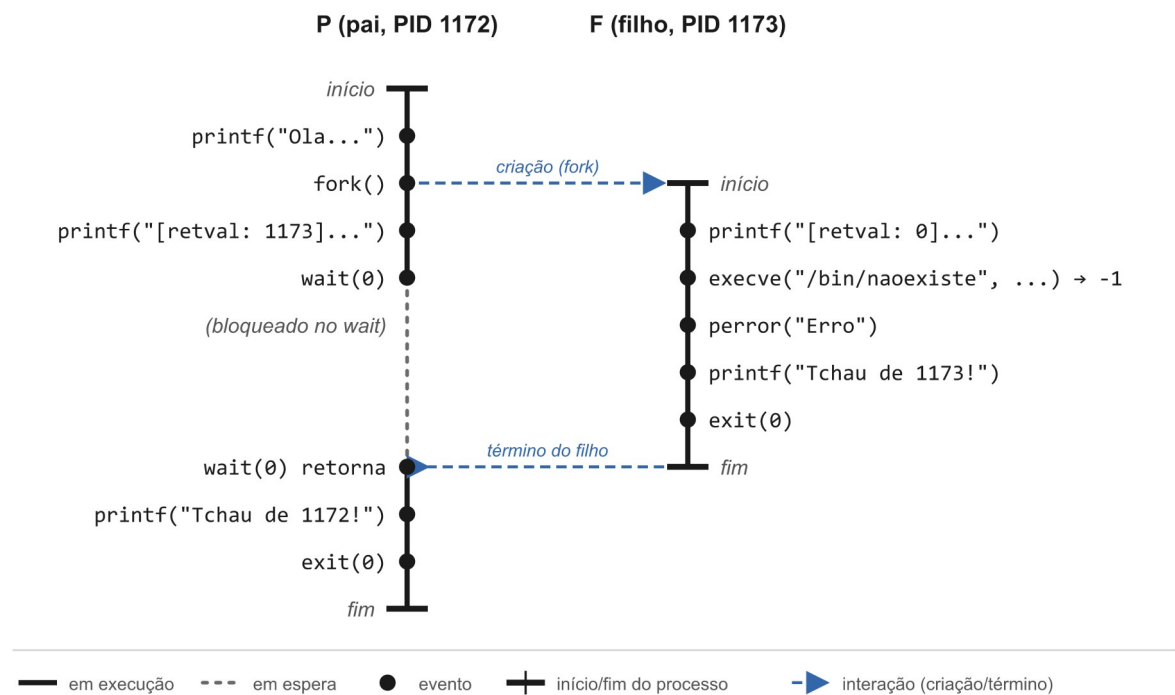
```

Asus@DESKTOP-B7L3I96 MSYS /c/Users/Asus/Documents/SO-Laboratório/Lab 2
gcc -Wall fork-execve-falha.c -o fork-execve-falha
./fork-execve-falha
Ola, sou o processo 1172
[retval: 1173] sou 1172, filho de 923
[retval: 0] sou 1173, filho de 1172
Erro: No such file or directory
Tchau de 1173!
Tchau de 1172!

```

Fonte: elaboração própria (2026).

Figura 7: Diagrama de tempo de fork-execve.c quando o programa a executar não existe



Fonte: elaboração própria (2026).

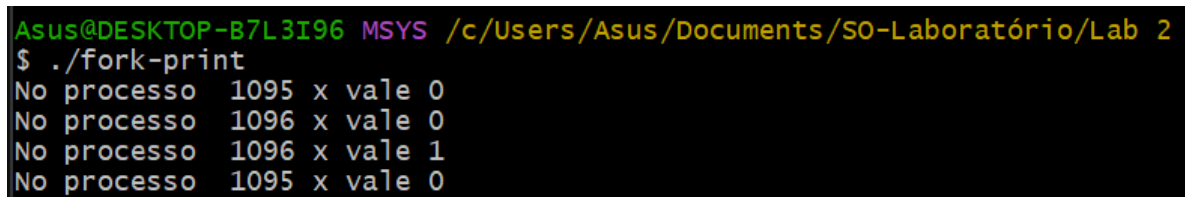
3 EXERCÍCIO 3: FORK-PRINT.C

3.1 ANÁLISE DO CÓDIGO

- a) `x = 0` antes do `fork()`; o filho recebe uma **cópia** de `x` (espaços de endereçamento separados; no Linux, via *copy-on-write*);
- b) Os dois processos imprimem `x vale 0`;
- c) O pai faz `x = 0` (não muda nada) e espera o filho; o filho faz `x++` (**na sua cópia**: `x = 1`) e dorme 5 s;
- d) O filho imprime `x vale 1` e termina; o pai, após o `wait`, imprime `x vale 0`;
- e) Conclusão: a alteração feita pelo filho **não é visível** ao pai, pois processos não compartilham memória. Para trocar dados seria preciso IPC (pipes, memória compartilhada, etc.).

Saída real (MSYS2):

Figura 8: Captura de tela da execução de `fork-print.c`

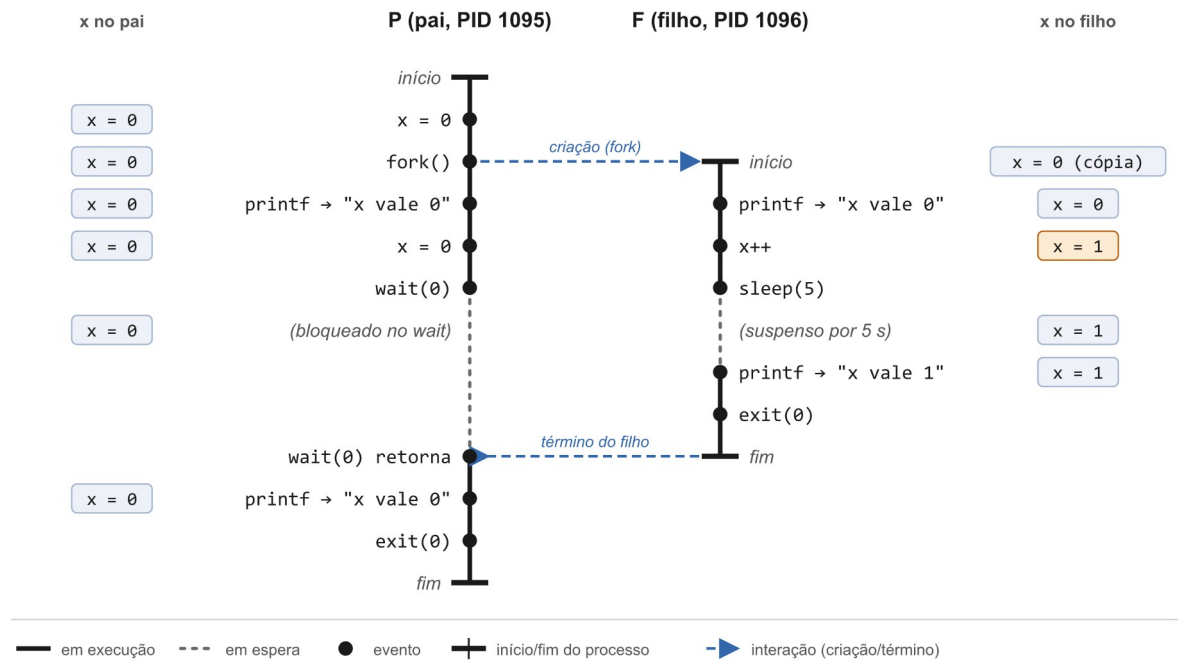


```
Asus@DESKTOP-B7L3I96 MSYS /c/Users/Asus/Documents/SO-Laboratório/Lab 2
$ ./fork-print
No processo 1095 x vale 0
No processo 1096 x vale 0
No processo 1096 x vale 1
No processo 1095 x vale 0
```

Fonte: elaboração própria (2026).

3.2 DIAGRAMA DE TEMPO (COM A EVOLUÇÃO DE X)

Figura 9: Diagrama de tempo de fork-print.c com a evolução da variável x



Fonte: elaboração própria (2026).

Evolução de x:

Tabela 1: Evolução do valor de x no pai e no filho

Momento	x no pai	x no filho
antes do fork()	0	inexistente
logo após o fork()	0	0 (cópia)
após x = 0 / x++	0	1
printf final	0	1

Fonte: elaboração própria (2026).